

MACHINA PARTY

プログラム説明資料



ジャンル: マルチプレイ 3D アクション/パーティーゲーム

開発環境: C# / Unity6 | 開発期間: 2026/4 ~ 制作中 (2人制作)

横浜デジタルアーツ専門学校 ゲーム科3年 石川原吉美

担当: ゲーム内容全般

C#

Unity6

NavMesh

CharacterController

MACHINA PARTY

開発体制・使用ツールについて

01 所要時間の概算

- ・ 開発開始から現在まで約13週間（2026年4月～）
- ・ 週3～4回×2～3時間ペースで継続中
- ・ 自身の実装時間目安 約100時間（制作中のため今後増加）

02 開発体制・担当範囲

- ・ 2人体制（自身+1名）
- ・ 自身の担当：ゲーム内容全般（GameManager・Tank関連システム・レースシステム等のプログラム実装）

03 学校のFW・AI活用範囲

- ・ 学校指定の共通フレームワークは不使用、ゼロから構築
- ・ 実装のエラー・不明点はAIに質問し参考にした
- ・ タイトル画面イラストはAI画像生成（ChatGPT）で制作
- ・ ゲームロジックの設計・実装は自身で実施

04 外部アセット・ミドルウェア

- ・ タイトル画面イラスト以外のグラフィックはフリー素材を使用し、Photoshopで加工
- ・ ミドルウェア・外部SDKの組み込みはなし

アピールポイント

01 GameManager 設計

ラウンド制御・生存判定・勝者判定を1クラスに集約。PlayerDataでCPU/Human・カラー・操作設定をメニューからゲームへ受け渡す。

02 TankMovement（移動）

CharacterControllerで3D移動を実装。Gamepad操作時はカメラ方向基準のTPS操作に自動切替。爆風ノックバックも加速度減衰で実装。

03 TankShooting（射撃）

ボタン長押しでチャージが増加し最大で自動発射。放物線計算でAIの着弾予測地点を算出。強化弾(特殊弾)システムも搭載。

04 TankAI（CPU制御）

NavMeshで最短経路を計算し敵を追跡(Seek)。近づきすぎると逃走(Flee)に遷移。射程・壁チェックで射撃タイミングを自律判定。

05 TankHealth（HP管理）

HPバーをSmoothDampでアニメーション。被弾後の無敵時間(点滅演出)シールドによるダメージ軽減。ノックバック無効化と連携。

06 レースシステム

LapCounterでチェックポイントの順番通過を管理。RaceSplitScreenCameraManagerが人数に応じて分割画面カメラを自動生成。

ゲームシステム — アーキテクチャ全体図

アーキテクチャ

Main Menu

GameUIHandler

メニュー・スロット・ポーズ管理

StartMenuSlot

プレイヤー設定UI

PauseMenu

ポーズ画面制御

TankMode Scene

GameManager

ラウンド進行・生存判定

TankManager

タンク個別管理

TankMovement

CharacterController移動

TankShooting

チャージ射撃

TankAI

NavMesh CPU AI

TankHealth

HP・無敵・シールド

CameraControl

全体追従カメラ

RaceMode Scene

CarMovement

カート移動・ドリフト

LapCounter

周回・チェックポイント

CheckPoint

CP通過判定

GoalTrigger

ゴール通過検知

SplitScreen

分割画面カメラ管理

GameManager 設計

ゲーム進行管理 — ラウンド制御・生存判定・タンク生成

ラウンド制御

GameLoop()コルーチン

RoundStarting→RoundPlaying→RoundEnding を順番に処理。
m_NumRoundsToWin勝した時点でシーンをリロードしゲーム終了。

生存判定

OneTankLeft()でactiveSelfなタンク数を毎フレーム計測。1体以下になったらRoundPlaying()を抜けてRoundEndingへ進む。

PlayerData

IsComputer・TankColor・UsedPrefab・ControllIndexをメニューからGameManagerに渡す内部クラス。プレイヤー数配列でまとめて受け取る。

SpawnAllTanks()

PlayerData配列の人数分Instantiateし、全生成後にSetup()で各TankManagerを初期化。カメラ追従対象(Transform[])も同時に設定。

// GameManager.cs 抜粋

```
private IEnumerator GameLoop()
{
    yield return StartCoroutine(
        RoundStarting());
    yield return StartCoroutine(
        RoundPlaying());
    yield return StartCoroutine(
        RoundEnding());

    if (m_GameWinner != null)
        SceneManager.LoadScene(0);
    else
        StartCoroutine(GameLoop());
}
```

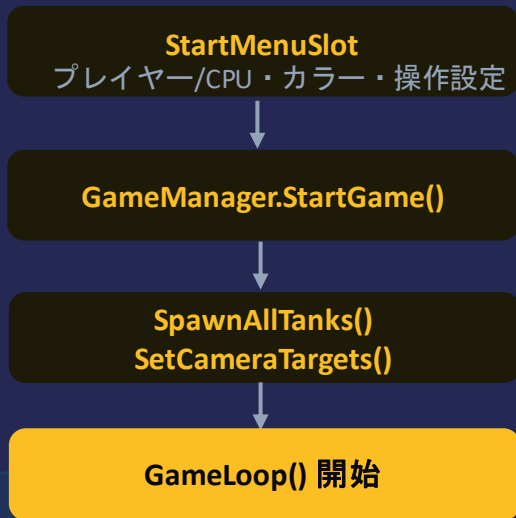
// 生存判定

```
private bool OneTankLeft()
{
    int numTanksLeft = 0;
    for (int i = 0;
        i < m_PlayerCount; i++)
    {
        if (m_SpawnPoints[i]
            .m_Instance.activeSelf)
            numTanksLeft++;
    }
    return numTanksLeft <= 1;
}
```

GameManager — ゲーム進行フロー図

MainMenu → GameScene (ラウンドループ)

設定フロー



ラウンドループ(5ラウンド先取の場合)



SceneManager
.LoadScene(0)

TankMovement — プレイヤー移動

操作・TPS制御・爆風ノックバック・エンジン音

CharacterController移動

Rigidbodyを使わずCharacterControllerで3D移動を実装。
stepOffset・slopeLimit・skinWidthを設定し段差・坂を自然に
乗り越える。

カメラ方向基準TPS操作

Gamepadまたはm_IsDirectControlがtrueのとき、
Camera.main.forward/rightを基準に入力ベクトルを変換。
LookRotation+RotateTowardsで滑らかに向きを転換。

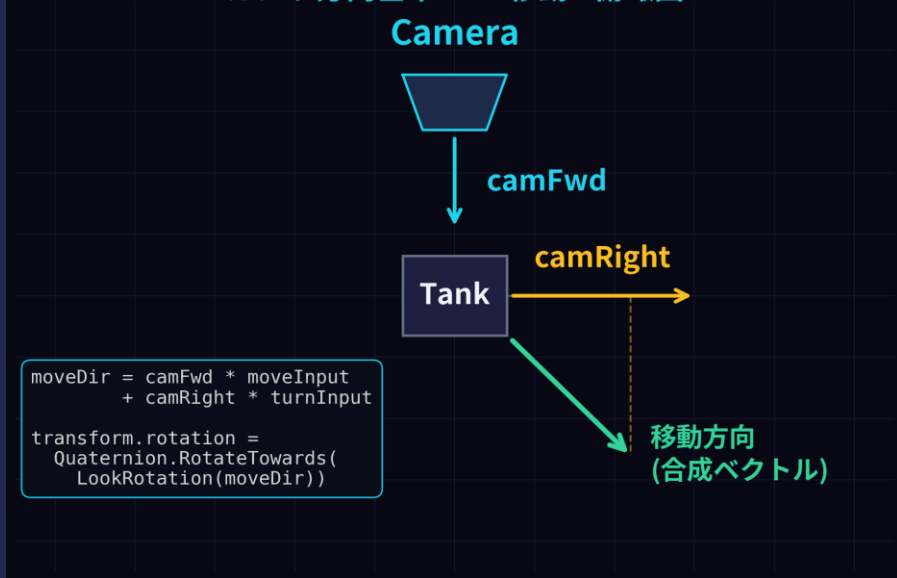
爆風ノックバック

AddExplosionForce()が爆発位置・距離・減衰係数から
m_ExplosionVelocityを計算し移動に加算。Lerpで徐々に減衰
し、無敵中は完全無効化。

エンジン音制御

移動入力の有無でm_EngineIdling/m_EngineDrivingを切り替え
再生。毎回ランダムなPitchを加えることで自然なエンジン
音を実現。

カメラ方向基準 TPS 移動 - 俯瞰図



```
// TankMovement.cs - カメラ方向基準 TPS
Vector3 camFwd = Camera.main.transform.forward;
camFwd.y = 0f; camFwd.Normalize();
moveDir = camFwd * m_MovementInputValue
          + camRight * m_TurnInputValue;
// Quaternion.RotateTowards() で滑らかに旋回
```

TankShooting — 射撃システム

チャージ射撃・放物線計算・特殊弾

チャージ射撃

ボタン長押しでm_CurrentLaunchForceが増加しUISライダーに反映。最大チャージで自動発射。ボタン離し・最大到達の2種類のトリガー。

放物線着弾計算

GetProjectilePosition()で発射速度・重力加速度から二次方程式を解き着弾予測位置を算出。TankAIが射程確認と狙い撃ちに利用する。

特殊弾（強化弾）

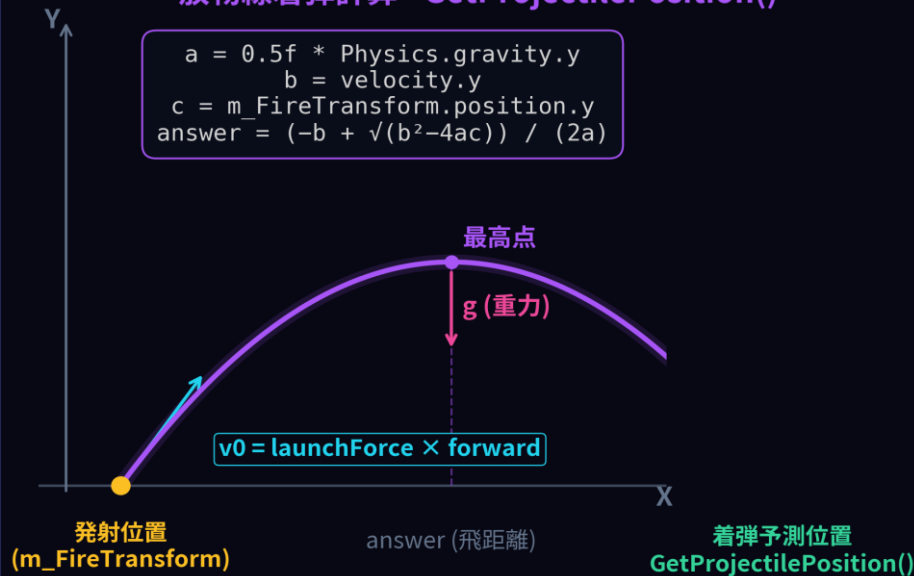
EquipSpecialShell()でm_SpecialShellMultiplierを設定。次の発射時にダメージを倍増させ、発射後は自動解除。HUDでアクティブ表示。

クールタイム

m_ShotCooldownTimerで発射後の連射を制限。AIはStartCharging()/StopCharging()APIで射撃を制御しHumanと同じFireパスを通る。

放物線着弾計算 - GetProjectilePosition()

```
a = 0.5f * Physics.gravity.y  
b = velocity.y  
c = m_FireTransform.position.y  
answer = (-b + sqrt(b2-4ac)) / (2a)
```



```
// TankShooting.cs - 放物線着弾計算  
float a = 0.5f * Physics.gravity.y;  
float b = velocity.y;  
float c = m_FireTransform.position.y;  
// 二次方程式で着弾時間 t を算出  
answer = (-b + Sqrt(b*b - 4*a*c)) / (2*a);
```


TankAI — CPU制御

NavMesh経路探索 ・ Seek/Flee状態機械 ・ 射撃タイミング判定

NavMesh経路探索

ランダム間隔(0.3~0.6秒)で全敵へのNavMeshPathを計算し最短経路の敵をターゲット確定。Cornerを順番に追いかけてながら追跡する。

Seek → Flee 遷移

敵との距離が3m未満かつ2秒以上継続するとFlee状態へ。逃走先を90~180度ランダム方向・5~20mでNavMesh計算し回避経路を生成。

射撃タイミング判定

射程内かつNavMesh.Raycastで壁なし→チャージ開始。着弾予測位置が敵距離に達しdot積>0.99(ほぼ正面)でStopCharging()して発射。

負荷分散

m_PathfindTimeをAwakeでRandom.Range(0.3f, 0.6f)に設定。複数CPUの経路探索を時間的にバラつかせてフレームのスパイクを軽減。

TankAI — Seek / Flee 状態遷移図



```
// TankAI.cs - Seek/Flee 状態遷移
if (dist < 3f && closeTimer > 2f)
    m_CurrentState = State.Flee;
else if (navPathComplete)
    m_CurrentState = State.Seek;
// Vector3.Dot > 0.99 で StopCharging()→発射
```

TankHealth — HP管理

HP管理・無敵時間・シールド・HPバーアニメーション

HP管理

TakeDamage()でm_CurrentHealthを減算。HP0かつm_Dead=falseのときOnDeath()を呼び爆発エフェクト生成→SetActive(false)でラウンド終了を通知。

無敵時間・点滅演出

被弾後にInvincibleRoutine()コルーチンを起動。
m_BlinkInterval間隔でRenderer.enabledをON/OFFし点滅。
m_InvincibleTimeが経過すると解除。

シールド

m_HasShield有効時は受けるダメージを(1-m_ShieldValue)で軽減。PowerUpアイテムから付与・解除が可能。Equipで倍率も動的変更できる。

HPバーアニメーション

m_DisplayHealthをSmoothDampでm_CurrentHealthに追従させスライダーを更新。HP割合に応じて
m_FullHealthColor→m_ZeroHealthColorにColor.Lerpで変色。

```
// TankHealth.cs 抜粋
public void TakeDamage(float amount)
{
    if (m_IsInvincible) return;
    if (m_HasShield)
        amount *= (1f - m_ShieldValue);
    m_CurrentHealth -= amount;
    if (m_CurrentHealth <= 0f &&
        !m_Dead)
        OnDeath();
    else
    {
        if (m_InvincibleCoroutine != null)
            StopCoroutine(
                m_InvincibleCoroutine);
        m_InvincibleCoroutine =
            StartCoroutine(
                InvincibleRoutine());
    }
}

// HPバーUpdate
m_DisplayHealth = Mathf.Lerp(
    m_DisplayHealth, m_CurrentHealth,
    Time.deltaTime *
    m_HealthBarSmoothSpeed);
m_Slider.value = m_DisplayHealth;
m_FillImage.color = Color.Lerp(
    m_ZeroHealthColor,
    m_FullHealthColor,
    m_CurrentHealth / m_StartingHealth);
```

レースシステム

LapCounter（周回管理） ・ RaceSplitScreenCameraManager（分割画面）

チェックポイント順番管理

PassCheckPoint()に通過番号が渡され、currentCheckPointと一致する場合のみカウント。順番スキップは無効とし逆走・飛ばしを防ぐ設計。

初周フラグ

Z座標がfirstLapEnableZ(100f)に達するまでゴール通過を無効化。スタート直後にゴールラインを通過してしまう誤カウントを防ぐ。

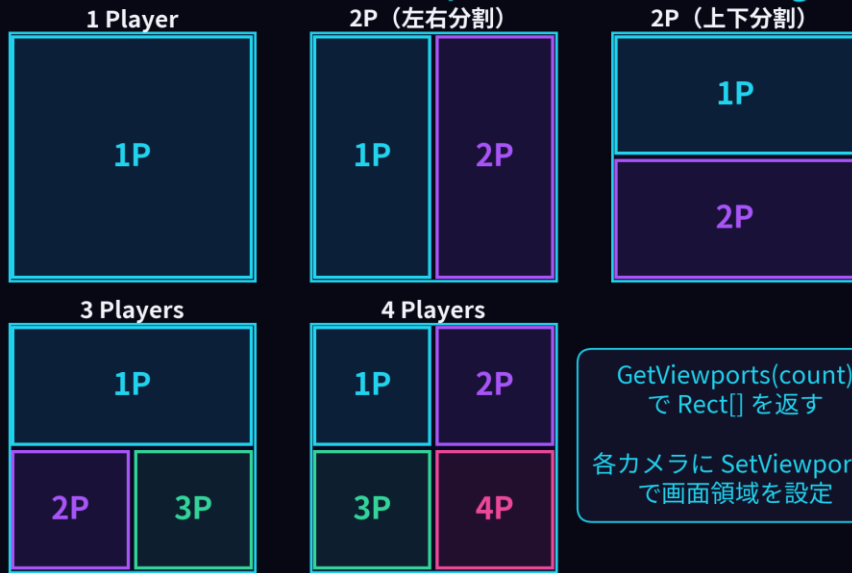
分割画面カメラ自動生成

Setup()でプレイヤー数分のRaceCameraFollowをInstantiate。isOnlineModeがtrueの場合は1画面固定。各カメラにTargetを割り当て独立追従。

レイアウト自動設定

2人時はVertical(左右)/Horizontal(上下)を選択可能。3～4人時は自動でViewport Rectを四分割配置。GetViewports()で人数ごとのRectを返す。

分割画面レイアウト - RaceSplitScreenCameraManager



```
// LapCounter - CP順番確認
if (checkPointNumber != currentCP) return;
currentCheckPoint++;
// RaceSplitScreen - Viewport自動設定
Rect[] vp = GetViewports(cameraCount);
for (int i=0; i<cameras.Length; i++) cameras[i].rect = vp[i];
```

現状のまとめ

● GameManager

ラウンド進行・生存判定・PlayerDataによるシーン間設定受け渡しを1クラスに集約し状態遷移を安全に制御

● TankMovement

CharacterControllerによる3D移動とGamepad時のTPS操作自動切替。爆風ノックバックも統合

● TankShooting

チャージ射撃・放物線着弾計算・特殊弾システムをHuman/CPU共通のFireパスで処理

● TankAI

NavMesh最短経路探索によるSeek追跡と、近接回避のFlee逃走を組み合わせた自律型CPUを実装

● レースシステム

LapCounterのCP順番管理と、RaceSplitScreenCameraManagerの人数対応分割画面でレースモードを構築中